GDB 程序调试方法说明

Revision History

Draft Date	Revision No.	Description
2024/06/25	V1.0	1. 初始版本。

目　　录

前　言 3
1 演示程序创建与编译 4
2 调试程序运行 5
3 程序调试 7
3.1 list 命令 7
3.2 break 命令 8
3.3 info breakpoints 命令 8
3.4 continue 命令 8
3.5 print 命令 9
3.6 quit 命令 11
更多帮助 12

前 言

本指导文档适用开发环境：

Windows 开发环境：Windows 7 64bit、Windows 10 64bit

虚拟机：VMware15.5.5

Linux 开发环境：Ubuntu20.04.6 64bit

U-Boot：U-Boot-2022.04

Kernel：Linux-5.15.71

LinuxSDK：Real-Time Edge Software 2.5

GDB 工具是 GNU 项目调试器，基于命令行使用。和其他的调试器一样，可使用 GDB 工具单步运行程序、单步执行、跳入/跳出函数、设置断点、查看变量等等，它是 UNIX/LINUX 操作系统下强大的程序调试工具。GDB 支持多种语言，包括 Ada、汇编、C/C++、D、Fortran、GO、Objective-C、OpenCL、Modula-2、Pascal 和 Rust。

GDB 具有一种“远程”模式，此种模式多半是在为嵌入式系统进行调试时使用，远程操作指的是：GDB 在一部机器内运行，而要进行调试的程序是在另一部机器上运行，支持 TCP/IP 协议传输的各种网络，在 TCP/IP 网络及协议上再加搭传输 GDB 专有的调试操作通信协议，如此便能进行远程调试。

1 演示程序创建与编译

在 Ubuntu 工作目录下执行如下命令，新建一个 GDB 调试演示程序。

Host# mkdir test

Host# cd test/

Host# vi test.c

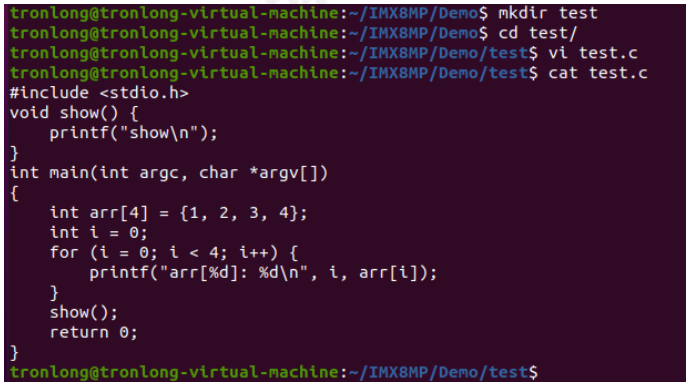

图 1

```
#include <stdio.h>
void show() {
    printf("show\n");
}
int main(int argc, char *argv[])
{
    int arr[4] = {1, 2, 3, 4};
    int i = 0;
    for (i = 0; i < 4; i++) {
        printf("arr[%d]: %d\n", i, arr[i]);
    }
    show();
    return 0;
}
```

执行如下命令，使能 SDK 交叉编译工具链并进行程序编译，编译完成将会生成 test 可执行文件。

Host# source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

Host# $CC -O0 -g test.c -o test

备注：编译时需添加-g 参数，生成的可执行文件才带有调试信息。

```
tronlong@tronlong-virtual-machine:~/IMX8MP/Demo/test$ source /home/tronlong/SDK/e
nvironment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/IMX8MP/Demo/test$ $CC -O0 -g test.c -o test
In file included from /home/tronlong/SDK/sysroots/armv8a-poky-linux/usr/include/b
its/libc-header-start.h:33,
                 from /home/tronlong/SDK/sysroots/armv8a-poky-linux/usr/include/s
tdio.h:27,
                 from test.c:1:
/home/tronlong/SDK/sysroots/armv8a-poky-linux/usr/include/features.h:412:4: warni
ng: #warning _FORTIFY_SOURCE requires compiling with optimization (-O) [-Wcpp]
  412 | #  warning _FORTIFY_SOURCE requires compiling with optimization (-O)
      |    ^~~~~~~
tronlong@tronlong-virtual-machine:~/IMX8MP/Demo/test$ ls
test  test.c
tronlong@tronlong-virtual-machine:~/IMX8MP/Demo/test$
```

图 2

评估板与 PC 机连接至同一个路由器后进行上电，将编译生成的可执行文件 test 拷贝至评估板文件系统任意目录下。

Target# ls

```
root@IMX8MP-Tronlong:~# ls
test
root@IMX8MP-Tronlong:~#
```

图 3

2 调试程序运行

执行如下命令，查看虚拟机的 IP 地址。

Host# ifconfig

```
tronlong@tronlong-virtual-machine:~/IMX8MP/Demo/test$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.31.225  netmask 255.255.255.0  broadcast 192.168.31.255
        inet6 fe80::28ef:f51e:a9dc:26a3  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:1c:48:23  txqueuelen 1000  (Ethernet)
        RX packets 252167  bytes 138040185 (138.0 MB)
        RX errors 0  dropped 2  overruns 0  frame 0
        TX packets 348294  bytes 445585749 (445.5 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 242  bytes 32015 (32.0 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 242  bytes 32015 (32.0 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

tronlong@tronlong-virtual-machine:~/IMX8MP/Demo/test$
```

图 4

在评估板文件系统 test 文件所在路径，执行如下命令运行 gdbserver 程序，开启监听。

备注：192.168.31.225 为虚拟机的 IP 地址；1234 为连接的端口号（自行设置）；test 为将要进行调试的程序。

Target# gdbserver 192.168.31.225:1234 test

```
root@IMX8MP-Tronlong:~# gdbserver 192.168.31.225:1234 test
Process /home/root/test created; pid = 1426
Listening on port 1234
```

图 5

在虚拟机 test 文件所在路径下，执行如下命令运行 GDB 调试程序。

Host# aarch64-poky-linux-gdb test

```
tronlong@tronlong-virtual-machine:~/IMX8MP/Demo/test$ aarch64-poky-linux-gdb test
GNU gdb (GDB) 11.2
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-pokysdk-linux --target=aarch64-poky-lin
ux".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from test...
(gdb)
```

图 6

执行如下命令，连接评估板。192.168.31.95 为评估板的 IP 地址，请以实际情况为准。

(gdb) target remote 192.168.31.95:1234

```
(gdb) target remote 192.168.31.95:1234
Remote debugging using 192.168.31.95:1234
Reading /lib/ld-linux-aarch64.so.1 from remote target...
warning: File transfers from remote targets can be slow. Use "set sysroot" to acc
ess files locally instead.
Reading /lib/ld-linux-aarch64.so.1 from remote target...
Reading symbols from target:/lib/ld-linux-aarch64.so.1...
Reading /lib/.debug/ld-linux-aarch64.so.1 from remote target...
Reading /lib/.debug/ld-linux-aarch64.so.1 from remote target...
Reading symbols from target:/lib/.debug/ld-linux-aarch64.so.1...
0x0000fffff7fd9cf0 in _start () from target:/lib/ld-linux-aarch64.so.1
(gdb)
```

图 7

同时，串口调试终端显示连接成功的相关信息。

```
root@IMX8MP-Tronlong:~# gdbserver 192.168.31.225:1234 test
Process /home/root/test created; pid = 783
Listening on port 1234
Remote debugging from host 192.168.31.225, port 42936
```

图 8

3 程序调试

本章节主要演示 GDB 调试的几个常用命令的使用方法。

3.1 list 命令

用于列出源码，简写：l。

```
(gdb) l
1       #include <stdio.h>
2       void show() {
3           printf("show\n");
4       }
5       int main(int argc, char *argv[])
6       {
7           int arr[4] = {1, 2, 3, 4};
8           int i = 0;
9           for (i = 0; i < 4; i++) {
10              printf("arr[%d]: %d\n", i, arr[i]);
(gdb)
11          }
12          show();
13          return 0;
14      }
(gdb)
```

图 9

3.2 break 命令

用于设置断点，简写：b。

分别在 main 函数和第 10 行设置断点。

```
(gdb) b main
Breakpoint 1 at 0xaaaaaaaa0954: file test.c, line 6.
(gdb) b 10
Breakpoint 2 at 0xaaaaaaaa0994: file test.c, line 10.
(gdb)
```

图 10

3.3 info breakpoints 命令

用于显示断点的状态，简写：info b。

```
(gdb) info b
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x0000aaaaaaaa0954 in main at test.c:6
2       breakpoint     keep y   0x0000aaaaaaaa0994 in main at test.c:10
(gdb)
```

图 11

3.4 continue 命令

用于恢复程序执行，简写：c。此命令使 GDB 恢复程序的执行，直至断点触发或程序结束。

```
(gdb) c
Continuing.
Reading /lib/libc.so.6 from remote target...
Reading /lib/.debug/libc.so.6 from remote target...
Reading /lib/.debug/libc.so.6 from remote target...

Breakpoint 1, main (argc=1, argv=0xffffffffc08) at test.c:6
6       {
(gdb) c
Continuing.

Breakpoint 2, main (argc=1, argv=0xffffffffc08) at test.c:10
10              printf("arr[%d]: %d\n", i, arr[i]);
(gdb) c
Continuing.

Breakpoint 2, main (argc=1, argv=0xffffffffc08) at test.c:10
10              printf("arr[%d]: %d\n", i, arr[i]);
(gdb) c
Continuing.

Breakpoint 2, main (argc=1, argv=0xffffffffc08) at test.c:10
10              printf("arr[%d]: %d\n", i, arr[i]);
(gdb) c
Continuing.

Breakpoint 2, main (argc=1, argv=0xffffffffc08) at test.c:10
10              printf("arr[%d]: %d\n", i, arr[i]);
(gdb) c
Continuing.
[Inferior 1 (process 783) exited normally]
(gdb)
```

图 12

以上调试过程在第二个断点中，每输入一次 c 命令，将会依次打印 arr[]数组的元素值，然后程序执行结束。同时，串口调试终端打印相关信息。

```
root@IMX8MP-Tronlong:~# gdbserver 192.168.31.225:1234 test
Process /home/root/test created; pid = 783
Listening on port 1234
Remote debugging from host 192.168.31.225, port 42936
arr[0]: 1
arr[1]: 2
arr[2]: 3
arr[3]: 4
show

Child exited with status 0
root@IMX8MP-Tronlong:~#
```

图 13

3.5 print 命令

用于打印变量或者变量地址，简写：p。

设置断点后，可依次打印当前变量 i 的值。

```
(gdb) target remote 192.168.31.56:1234
Remote debugging using 192.168.31.56:1234
Reading /lib/ld-linux-aarch64.so.1 from remote target...
warning: File transfers from remote targets can be slow. Use "set sysroot" to acces
s files locally instead.
Reading /lib/ld-linux-aarch64.so.1 from remote target...
Reading symbols from target:/lib/ld-linux-aarch64.so.1...
Reading /lib/.debug/ld-linux-aarch64.so.1 from remote target...
Reading /lib/.debug/ld-linux-aarch64.so.1 from remote target...
Reading symbols from target:/lib/.debug/ld-linux-aarch64.so.1...
0x0000ffff7fd9cf0 in _start () from target:/lib/ld-linux-aarch64.so.1
(gdb) b main
Breakpoint 1 at 0xaaaaaaaa0954: file test.c, line 6.
(gdb) b 10
Breakpoint 2 at 0xaaaaaaaa0994: file test.c, line 10.
(gdb) info b
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x0000aaaaaaaa0954 in main at test.c:6
2       breakpoint     keep y   0x0000aaaaaaaa0994 in main at test.c:10
(gdb) c
Continuing.
Reading /lib/libc.so.6 from remote target...
Reading /lib/.debug/libc.so.6 from remote target...
Reading /lib/.debug/libc.so.6 from remote target...

Breakpoint 1, main (argc=1, argv=0xffffffffc08) at test.c:6
6       {
(gdb) c
Continuing.

Breakpoint 2, main (argc=1, argv=0xffffffffc08) at test.c:10
10              printf("arr[%d]: %d\n", i, arr[i]);
(gdb) p i
$1 = 0
```

图 14

```
(gdb) c
Continuing.

Breakpoint 2, main (argc=1, argv=0xffffffffc08) at test.c:10
10              printf("arr[%d]: %d\n", i, arr[i]);
(gdb) p i
$2 = 1
(gdb) c
Continuing.

Breakpoint 2, main (argc=1, argv=0xffffffffc08) at test.c:10
10              printf("arr[%d]: %d\n", i, arr[i]);
(gdb) p i
$3 = 2
(gdb) c
Continuing.

Breakpoint 2, main (argc=1, argv=0xffffffffc08) at test.c:10
10              printf("arr[%d]: %d\n", i, arr[i]);
(gdb) p i
$4 = 3
(gdb) c
Continuing.
[Inferior 1 (process 1510) exited normally]
(gdb)
```

图 15

以上调试过程在每处断点中，使用 print 命令打印变量 i 的值。同时，串口调试终端打印相关信息。

```
root@IMX8MP-Tronlong:~# gdbserver 192.168.31.225:1234 test
Process /home/root/test created; pid = 1510
Listening on port 1234
Remote debugging from host 192.168.31.225, port 52642
arr[0]: 1
arr[1]: 2
arr[2]: 3
arr[3]: 4
show

Child exited with status 0
root@IMX8MP-Tronlong:~#
```

图 16

3.6 quit 命令

退出 GDB 模式，简写：q。当程序调试完成后，使用 quit 命令退出 GDB 调试。

```
(gdb) p i
$4 = 3
(gdb) c
Continuing.
[Inferior 1 (process 1510) exited normally]
(gdb) q
tronlong@tronlong-virtual-machine:~/IMX8MP/Demo/test$
```

图 17

同时，串口调试终端打印相关信息。

```
root@IMX8MP-Tronlong:~# gdbserver 192.168.31.225:1234 test
Process /home/root/test created; pid = 1519
Listening on port 1234
Remote debugging from host 192.168.31.225, port 36240
arr[0]: 1
arr[1]: 2
arr[2]: 3
arr[3]: 4
show

Child exited with status 0
root@IMX8MP-Tronlong:~#
```

图 18

更多帮助

销售邮箱：sales@tronlong.com

技术邮箱：support@tronlong.com

创龙总机：020-8998-6280

技术热线：020-3893-9734

创龙官网：www.tronlong.com

技术论坛：www.51ele.net

官方商城：tronlong.tmall.com